Bulk Collection: Rejected Rows & LIMIT

SAVE EXCEPTIONS, SQL%BULK_EXCEPTIONS & the LIMIT clause · Oracle 19c

Two independent controls make bulk processing production-safe: **LIMIT** caps how many rows a single `BULK COLLECT` pulls into memory (batching), and **SAVE EXCEPTIONS** lets a `FORALL` continue past rejected rows and report them afterward.

1. The LIMIT Clause — Batching Fetches

An unbounded `BULK COLLECT` can exhaust PGA memory. Fetch in fixed batches with a cursor + `LIMIT`:

```
DECLARE
  CURSOR c IS SELECT * FROM emp_src;
  TYPE t_emp IS TABLE OF emp_src%ROWTYPE;
  l_emp   t_emp;
  c_limit CONSTANT PLS_INTEGER := 500;         -- batch size
BEGIN
  OPEN c;
  LOOP
    FETCH c BULK COLLECT INTO l_emp LIMIT c_limit;
    EXIT WHEN l_emp.COUNT = 0;               -- nothing left

    FORALL i IN 1 .. l_emp.COUNT
      INSERT INTO emp_tgt VALUES l_emp(i);

    COMMIT;                                 -- commit per batch (optional)
    EXIT WHEN c%NOTFOUND;                    -- last partial batch processed
  END LOOP;
  CLOSE c;
END;
/
```

Put `EXIT WHEN l_emp.COUNT = 0` **after** the FETCH and process the batch **before** `EXIT WHEN c%NOTFOUND` — otherwise the final partial batch is skipped. Typical LIMIT values: 100–1000.

2. SAVE EXCEPTIONS — Continue Past Rejected Rows

By default a `FORALL` aborts on the first DML error and rolls back the whole statement. `SAVE EXCEPTIONS` makes it process every element, collecting failures into `SQL%BULK_EXCEPTIONS`.

```
DECLARE
  TYPE t_emp IS TABLE OF emp_src%ROWTYPE;
  l_emp t_emp;
  l_err_cnt PLS_INTEGER;
  dml_errors EXCEPTION;
  PRAGMA EXCEPTION_INIT(dml_errors, -24381);      -- ORA-24381
BEGIN
  SELECT * BULK COLLECT INTO l_emp FROM emp_src;

  FORALL i IN 1 .. l_emp.COUNT SAVE EXCEPTIONS
    INSERT INTO emp_tgt VALUES l_emp(i);

  COMMIT;
EXCEPTION
  WHEN dml_errors THEN
    l_err_cnt := SQL%BULK_EXCEPTIONS.COUNT;
    DBMS_OUTPUT.put_line('Rejected rows: ' || l_err_cnt);

    FOR j IN 1 .. l_err_cnt LOOP
      DBMS_OUTPUT.put_line(
        'Index '  || SQL%BULK_EXCEPTIONS(j).error_index ||
        ' ORA-'  || SQL%BULK_EXCEPTIONS(j).error_code ||
        ' : '    || SQLERRM(-SQL%BULK_EXCEPTIONS(j).error_code));
    END LOOP;
    COMMIT;                                       -- good rows already applied
END;
/
```

SQL%BULK_EXCEPTIONS structure

Field	Meaning
`.COUNT`	Number of rejected rows
`(j).error_index`	Collection subscript that failed (map back to your data)
`(j).error_code`	Positive Oracle error number; use `SQLERRM(-code)` for text

3. Combine LIMIT + SAVE EXCEPTIONS (production pattern)

```
DECLARE
  CURSOR c IS SELECT * FROM emp_src;
  TYPE t_emp IS TABLE OF emp_src%ROWTYPE;
  l_emp t_emp;
  dml_errors EXCEPTION;
  PRAGMA EXCEPTION_INIT(dml_errors, -24381);
BEGIN
  OPEN c;
  LOOP
    FETCH c BULK COLLECT INTO l_emp LIMIT 500;
    EXIT WHEN l_emp.COUNT = 0;
    BEGIN
      FORALL i IN 1 .. l_emp.COUNT SAVE EXCEPTIONS
        INSERT INTO emp_tgt VALUES l_emp(i);
    EXCEPTION
      WHEN dml_errors THEN
        FOR j IN 1 .. SQL%BULK_EXCEPTIONS.COUNT LOOP
          INSERT INTO load_error_log(err_code, err_msg, key_value)
          VALUES (SQL%BULK_EXCEPTIONS(j).error_code,
                  SQLERRM(-SQL%BULK_EXCEPTIONS(j).error_code),
                  l_emp(SQL%BULK_EXCEPTIONS(j).error_index).id);
        END LOOP;
    END;
    COMMIT;
    EXIT WHEN c%NOTFOUND;
  END LOOP;
  CLOSE c;
END;
/
```

Key points

- `LIMIT` = memory control (batch size); `SAVE EXCEPTIONS` = error tolerance. They solve different problems — use both.
- `error_code` is returned **positive**; negate it for `SQLERRM`.
- With batching, wrap each batch's FORALL in its own `BEGIN/EXCEPTION` so one bad batch doesn't stop the whole load.
- For pure data loads, the SQL `LOG ERRORS` clause (see the Log Tables guide) is often simpler than SAVE EXCEPTIONS.